

EventHub - Master Plan

From poster to gate: the complete college event platform

Practice project | React + Spring Boot + MySQL

12-week milestone plan with 5 signature features

Owner: Madhavan | Version 2.0 - September 2026

1. The Big Idea

Most "event booking" projects on GitHub stop at **list events, then book, then pay**. They look the same, and recruiters have seen hundreds of them.

EventHub covers the complete life of a college event, from the day a club plans it to the day students download their participation certificate. Every stage solves a real problem that colleges have today.

PLAN	APPROVE	PUBLISH	BOOK	WAITLIST	GATE DAY	AFTER
Club creates a draft	-> Admin reviews & approves	-> Students discover & filter	-> Seat held 10 min, then pay	-> Sold out? join queue auto-offer	-> Volunteer scans QR ticket	-> Certificate PDF + feedback + analytics

Tagline: "EventHub - from poster to gate in one app."

Why this plan is different

Typical student project	EventHub (this plan)
Book button just reduces a number	Real booking engine : 10-minute seat hold, no overselling even with 100 users clicking at once
"Sold out" is a dead end	Smart waitlist automatically offers freed seats to the next student
Ticket = a row in a table	QR e-ticket scanned at the gate by volunteers on their phone
Project ends at payment	Auto-generated PDF certificates for students who actually attended
Anyone can post anything	Approval workflow : Draft, then Pending, then Published, like real platforms
Database created by hand	Flyway migrations , Docker, CI pipeline and automated tests - professional habits

2. The 5 Signature Features

These features make EventHub unique. Each one teaches an advanced concept that interviewers love to ask about.

Signature 1 - No-Oversell Booking Engine

- When a student clicks Book, the seat is **HELD for 10 minutes** while they pay.
- If they do not pay in time, a scheduled job releases the seat automatically.
- Uses **optimistic locking** (a version number on the event) so two students can never get the last seat.
- Proof: an automated test fires 100 parallel bookings at an event with 10 seats - exactly 10 succeed.

Real-life example: Like BookMyShow or IRCTC - seats are blocked while you pay and released if you leave the payment page.

You learn: @Transactional, @Version (optimistic locking), @Scheduled jobs, concurrency testing.

Signature 2 - Smart Waitlist

- When an event is full, students can **join the waitlist** and see their position (for example "You are #3").
- When someone cancels, the first person in the queue gets an **offer** (email + notification) valid for 30 minutes.
- If they do not accept in time, the offer moves to the next person automatically.

You learn: queues (FIFO), state machines, scheduled jobs, email with Spring Mail.

Signature 3 - QR E-Tickets & Gate Check-in

- Every confirmed booking gets a **unique QR code** ticket (ZXing library).
- Organizers assign **Volunteers**. A volunteer opens the Scanner page on their phone and scans tickets with the camera.
- The scanner shows a big green VALID or red ALREADY USED / INVALID, and a **live check-in counter**.

You learn: UUID tokens, image generation, a new role (VOLUNTEER), idempotent APIs, camera access in React.

Signature 4 - Attendance Certificates

- After the event is marked COMPLETED, every student who was **actually checked in** can download a PDF certificate.
- Each certificate has a unique certificate number and a **public verify link** - anyone can confirm it is genuine.

You learn: PDF generation (OpenPDF), file download APIs, public vs secured endpoints.

Real-life example: Students need participation certificates for their resumes and for college credits. Today clubs make them by hand in Canva - EventHub does it automatically.

Signature 5 - Approval Workflow & Organizer Analytics

- Organizers create events as **DRAFT** and submit them. The admin **approves or rejects with a reason**.
- Organizers get a dashboard with charts: bookings per day, revenue, waitlist size, check-in rate, average feedback rating.

You learn: enum state machines, role-based rules, JPQL aggregate queries, caching, charts (Recharts).

3. Scope - What We Build and When

Scope is split so that the project is **always working and demo-able**, even if you stop early.

Priority	Features
MUST (core)	Event catalogue with search, filter and pagination; clubs and tags; register/login with JWT; roles; booking with seat hold; My Tickets; Stripe payment; admin approval
SHOULD (signature)	Smart waitlist; QR tickets and scanner; certificates; organizer analytics; email notifications; feedback and ratings
COULD (stretch)	Team registration for hackathons; early-bird pricing and coupon codes; "Recommended for you" by interest tags; Add to Google Calendar (.ics); installable mobile app (PWA); Tamil language option
WON'T (this version)	Real money payments, a native mobile app, multi-college support

4. Users & Permissions

Action	Guest	Student	Volunteer	Organizer	Admin
Browse and search events	Yes	Yes	Yes	Yes	Yes
Book / cancel / join waitlist	-	Yes	Yes	Yes	-
Download ticket and certificate	-	Yes	Yes	Yes	-
Scan tickets at the gate	-	-	Own club	Own club	Yes
Create / edit events	-	-	-	Own club	Yes
Assign volunteers	-	-	-	Own club	Yes
Approve / reject events	-	-	-	-	Yes
See analytics	-	-	-	Own club	All
Manage users and clubs	-	-	-	-	Yes

Roles are **per club** for Organizer and Volunteer: Priya can be an Organizer in the Coding Club and just a Student everywhere else. This is a realistic design and a great interview talking point.

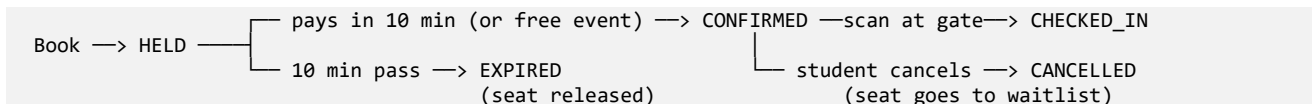
5. State Machines - the Heart of the System

Instead of random true/false flags, EventHub uses clear **states**. Only the arrows shown below are allowed; anything else is rejected with an error.

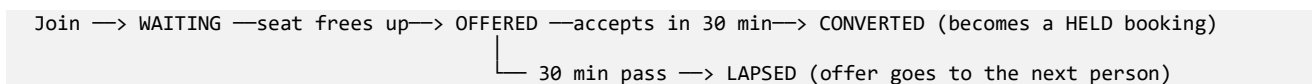
Event lifecycle



Booking lifecycle



Waitlist lifecycle



6. Database Design

```

users -1:1- profiles
users -M:N- roles (STUDENT, ADMIN)
clubs -1:N- events -M:N- tags
events -1:N- bookings -N:1- users
events -1:N- waitlist_entries
events -1:N- feedback
audit_logs (who did what, when)

clubs -1:N- club_members -N:1- users
                (role: ORGANIZER / VOLUNTEER)
bookings -1:1- payments
bookings -1:1- certificates
users -1:N- notifications

```

Table	Important columns
users	id, name, email (unique), password_hash, enabled, created_at
profiles	user_id, phone, department, year, interests
clubs	id, name, description, logo_url
club_members	club_id, user_id, member_role (ORGANIZER / VOLUNTEER)
events	id, club_id, title, description, venue, starts_at, ends_at, price, capacity, seats_available, status, reject_reason, version
bookings	id, event_id, user_id, status, ticket_code (UUID) , hold_expires_at, checked_in_at, checked_in_by
payments	id, booking_id, stripe_session_id, amount, status, paid_at
waitlist_entries	id, event_id, user_id, status, offer_expires_at, created_at (the queue order)
certificates	id, booking_id, certificate_no (unique), issued_at
feedback	id, event_id, user_id, rating (1-5), comment
notifications	id, user_id, message, read, created_at
audit_logs	id, actor_id, action, entity, entity_id, created_at

Rule: tables are created with **Flyway migration scripts** (V1__init.sql, V2__waitlist.sql, ...). Never change the database by hand - exactly how real companies work.

7. Architecture & Tech Decisions

React app (Vite)		Spring Boot 3 (Java 21)	MySQL 8
Pages + components	— REST / JSON —>	Controllers (web layer)	
Redux Toolkit store		Services (business rules, state machines)	
Axios client	<— JWT cookie —	Repositories (Spring Data JPA)	— Flyway —> tables
QR scanner (camera)		Schedulers (hold expiry, waitlist, reminders)	
		Integrations: Stripe Mail QR (ZXing) PDF (OpenPDF)	

Decision	Choice	Why
Backend	Java 21 + Spring Boot 3 (Web, Data JPA, Security, Validation, Cache, Mail, Actuator)	Industry standard, matches the course
Structure	Package by feature (event/, booking/, waitlist/...) not by layer	Easier to navigate as the project grows
Database	MySQL 8 in Docker + Flyway migrations	Repeatable setup on any laptop
Auth	JWT stored in an httpOnly cookie + Spring Security CSRF token	Safer than localStorage; practises the CSRF lesson
Frontend	React + Vite, React Router, Redux Toolkit, Tailwind CSS, Axios	Matches the course
Special libraries	ZXing (QR), OpenPDF (certificates), html5-qrcode (camera scanner), Recharts (charts)	Powers the signature features
Payments	Stripe Checkout (test mode) + webhook + Stripe CLI locally	Real payment flow without real money
Email	Spring Mail + Mailpit (local fake inbox in Docker)	See every email without spamming anyone
Testing	JUnit 5, Mockito, Testcontainers (real MySQL in tests), React Testing Library	Confidence that it really works
CI	GitHub Actions - build and test on every push	Green tick on GitHub impresses recruiters

Folder structure

```

02_EventHub_Practice_Project/
├── backend/src/main/java/com/eventhub/
│   ├── common/      (errors, base classes, security config, audit)
│   ├── auth/        (register, login, JWT)
│   ├── club/        (clubs, members, roles)
│   ├── event/       (catalogue, approval workflow)
│   ├── booking/     (seat hold, tickets, check-in)
│   ├── payment/     (Stripe checkout + webhook)
│   ├── waitlist/    (queue, offers)
│   ├── certificate/ (PDF + verify)
│   ├── feedback/    (ratings)
│   ├── analytics/   (dashboard queries)
│   └── notification/ (email + in-app)
├── backend/src/main/resources/db/migration/ (Flyway V1__..., V2__...)
├── frontend/src/
│   ├── features/    (auth, events, booking, waitlist, scanner, dashboard)
│   ├── components/ (shared UI: Button, Card, Modal, Navbar)
│   ├── app/        (Redux store, router, axios client)
│   └── pages/
├── docker-compose.yml (MySQL + Mailpit)
└── .github/workflows/ci.yml

```

8. Milestone Plan (12 weeks + foundations)

Each milestone ends with a **working version you can demo**, tagged in Git (v0.1, v0.2 ... v1.0). Plan for 1.5-2 hours per day.

M0 - Warm-up: EventHub in Plain Java (2-3 weeks, before Week 1)

Goal: Build the core booking rules as a console app in plain Java first - the logic you later move into Spring Boot.

Backend

- Classes: Event, Student, Booking, Waitlist; enum BookingStatus
- Book a seat, cancel, "Sold out" custom exception
- Waitlist with a Queue - cancelling gives the seat to the next person
- Menu-driven console: 1. List 2. Book 3. Cancel 4. Show waitlist

Frontend

- JavaScript warm-up: an HTML page that lists events from a JSON array with a search box

Concepts you learn: OOP, collections (List, Map, Queue), exceptions, enums, clean methods.

Demo / done when: The console app runs and handles "sold out" and waitlist correctly. Pushed to GitHub.

Interview question to practise: "What is the difference between a List and a Queue, and when would you use each?"

M1 - Skeleton & Pipeline (v0.1) (Week 1)

Goal: Both apps run, talk to each other, and GitHub automatically tests every push.

Backend

- Spring Boot project (start.spring.io) with the chosen dependencies
- docker-compose with MySQL + Mailpit
- Flyway V1 migration: users, clubs, events, tags
- Health endpoint + Swagger UI
- GitHub Actions workflow: build + test

Frontend

- Vite React app, Tailwind, Router, layout with Navbar and Footer, dark mode toggle
- Axios client that calls the health endpoint

Concepts you learn: Project setup, Maven, Docker, Flyway, CI/CD basics.

Demo / done when: The React page shows "Backend: UP". GitHub shows a green tick.

Interview question to practise: "Why use database migrations instead of letting Hibernate create tables?"

M2 - Event Catalogue (v0.2) (Weeks 2-3)

Goal: Anyone can browse, search and filter real events from the database.

Backend

- Entities + repositories: Club, Event, Tag (Many-to-Many)
- DTOs, validation, global exception handler with clean JSON errors
- Search by text, filter by tag/club/date, sort, **pagination**

- Seed data: 5 clubs, 20 events (Flyway V2)

Frontend

- Home page (upcoming + trending), Events page with filter sidebar and pagination
- Event details page, club page
- Loading skeletons, empty states, 404 page

Concepts you learn: REST design, DTO pattern, JPA relationships, Specifications / query methods, pagination.

Demo / done when: Filtering "Tech" + "this week" shows the correct events, page by page.

Interview question to practise: "Why should you never return JPA entities directly from a controller?"

M3 - Identity, Roles & Approval Workflow (v0.3) (Weeks 4-5)

Goal: Secure login, per-club roles, and events go through admin approval.

Backend

- Register / login / logout; BCrypt; JWT in httpOnly cookie; CSRF token
- Roles STUDENT / ADMIN + club roles ORGANIZER / VOLUNTEER
- Organizer: create, edit, submit event; Admin: approve / reject with reason
- Event state machine with allowed transitions only; audit log entries

Frontend

- Login and register pages with form validation
- Protected routes by role; "My Club" organizer area; admin approval queue page
- Auth state in Redux

Concepts you learn: Spring Security filter chain, JWT, authentication vs authorization, CSRF, enums as state machines.

Demo / done when: A student gets 403 on admin pages. A rejected event returns to DRAFT with the reason shown.

Interview question to practise: "Explain the difference between authentication and authorization, with an example from your project."

M4 - Booking Engine (v0.4) (Weeks 6-7)

Goal: Students can book free events safely - no overselling, ever.

Backend

- Booking entity with HELD / CONFIRMED / EXPIRED / CANCELLED
- **@Version optimistic locking** on Event; retry on conflict
- @Scheduled job every minute: expire holds, release seats
- Free events confirm immediately; cancel booking
- **Concurrency test:** 100 threads, 10 seats, exactly 10 bookings

Frontend

- Book button with countdown timer "Seat held for 09:59"
- Cart / checkout page (Redux)
- My Tickets page: upcoming, past, cancelled

Concepts you learn: @Transactional, isolation, optimistic locking, schedulers, multithreaded testing.

Demo / done when: Run the concurrency test live - it passes. The countdown expires and the seat comes back.

Interview question to practise: "Two users click Book on the last seat at the same moment. What happens in your system?"

M5 - Payments (v0.5) (Week 8)

Goal: Paid events go through Stripe; bookings confirm only after a verified payment.

Backend

- Create a Stripe Checkout session from a HELD booking
- **Webhook** endpoint with signature verification (the source of truth)
- Idempotency: the same webhook twice never double-confirms
- Payments table; release the seat if payment fails or expires

Frontend

- Pay button redirects to Stripe; success and cancelled pages
- Payment status on My Tickets

Concepts you learn: Third-party integration, webhooks, idempotency, secrets in environment variables, profiles.

Demo / done when: Test card 4242 4242 4242 4242 confirms the booking; the declined test card leaves it HELD and then EXPIRED.

Interview question to practise: "Why do you trust the webhook and not the success page redirect?"

M6 - Smart Waitlist & Notifications (v0.6) (Week 9)

Goal: Sold-out events have a fair, automatic queue.

Backend

- Join / leave waitlist; position calculation
- On cancellation or expiry: offer the seat to the next WAITING entry
- Offer expiry job (30 min) moves to the next person
- Emails via Spring Mail (Mailpit locally) + in-app notifications
- Reminder email the day before the event (@Scheduled cron)

Frontend

- "Join waitlist - you are #3" on sold-out events
- Notification bell with unread count
- Accept-offer page with countdown

Concepts you learn: Queues, event-driven thinking, scheduled cron jobs, email templates.

Demo / done when: Cancel a booking and watch the #1 waitlisted student receive an offer email in Mailpit.

Interview question to practise: "How do you make sure the waitlist is fair when two seats free up at once?"

M7 - Gate Day: QR Tickets & Check-in (v0.7) (Week 10)

Goal: Tickets become QR codes and volunteers check people in from their phones.

Backend

- QR image endpoint for each confirmed ticket (ZXing, contains the ticket_code only)

- Check-in API: VALID / ALREADY_CHECKED_IN / INVALID / WRONG_EVENT
- Only volunteers or organizers of that club can scan
- Live counter: checked in / total

Frontend

- Ticket page with a large QR code (and a download button)
- **Scanner page** using the phone camera (html5-qrcode), big green / red result screen, vibration
- Live check-in counter for organizers

Concepts you learn: Secure tokens (UUID), idempotent operations, image responses, mobile-first UI.

Demo / done when: Scan a ticket on a phone - green. Scan it again - red "Already used".

Interview question to practise: "Why does the QR code contain a random token instead of the booking id?"

M8 - After the Event: Certificates, Feedback & Analytics (v0.8) (Week 11)

Goal: Close the loop - certificates, ratings and dashboards.

Backend

- Mark event COMPLETED (automatic after end time)
- Certificate PDF (OpenPDF) for CHECKED_IN bookings only; unique number
- Public verify endpoint /verify/{certificateNo}
- Feedback (1-5 stars + comment), one per attendee
- Analytics JPQL queries; **@Cacheable** dashboard results

Frontend

- Download certificate button; public verify page
- Feedback form after the event
- Organizer dashboard with Recharts: bookings per day, revenue, check-in rate, rating
- Admin overview: totals across all clubs

Concepts you learn: File generation, aggregate queries, caching with eviction, data visualisation.

Demo / done when: Download a certificate, open the verify link and see "Genuine - issued to Ravi for Hackathon 2026".

Interview question to practise: "How does caching work in Spring and when must you evict the cache?"

M9 - Quality, Security & Launch (v1.0) (Week 12)

Goal: Production-ready, deployed and portfolio-ready.

Backend

- Unit tests (services) + integration tests with Testcontainers - aim for 70% coverage on services
- Rate limiting on login; secure Actuator and Swagger (ADMIN only)
- dev / prod profiles; all secrets from environment variables
- Deploy backend (Render / Railway) + managed MySQL

Frontend

- React Testing Library tests for key components
- Accessibility and mobile check on every page
- Deploy frontend (Vercel / Netlify)

Concepts you learn: Testing pyramid, security hardening, deployment, configuration management.

Demo / done when: Live URL works end to end: register, book, pay, get QR, scan, certificate.

Interview question to practise: "Walk me through how your application is deployed and configured in production."

9. Page Map

Route	Page	Who
/	Home - upcoming and trending events	Everyone
/events, /events/:id	Catalogue with filters; event details	Everyone
/clubs/:id	Club profile with its events	Everyone
/login, /register	Authentication	Guest
/checkout/:bookingId	Seat hold countdown and payment	Student
/me/tickets, /me/tickets/:id	My Tickets; QR ticket view	Student
/me/waitlist, /me/certificates	Waitlist positions; certificates	Student
/club/:id/manage	Organizer area: events, volunteers, dashboard	Organizer
/scan/:eventId	Gate scanner	Volunteer / Organizer
/admin/approvals, /admin	Approval queue; admin overview	Admin
/verify/:certificateNo	Public certificate verification	Everyone

10. Key API Endpoints

Module	Endpoints
Auth	POST /api/auth/register · POST /api/auth/login · POST /api/auth/logout · GET /api/auth/me
Events	GET /api/events?q=&tag=&club=&from=&to=&page= · GET /api/events/{id} · POST/PUT /api/clubs/{clubId}/events · POST /api/events/{id}/submit
Admin	GET /api/admin/approvals · POST /api/admin/events/{id}/approve · POST /api/admin/events/{id}/reject
Bookings	POST /api/events/{id}/bookings (creates HELD) · GET /api/me/bookings · POST /api/bookings/{id}/cancel
Payments	POST /api/bookings/{id}/checkout-session · POST /api/webhooks/stripe
Waitlist	POST/DELETE /api/events/{id}/waitlist · GET /api/me/waitlist · POST /api/waitlist/{id}/accept
Tickets	GET /api/bookings/{id}/qr (image) · POST /api/events/{id}/check-in {ticketCode}
Certificates	GET /api/bookings/{id}/certificate (PDF) · GET /api/public/certificates/{no}
Feedback	POST /api/events/{id}/feedback · GET /api/events/{id}/feedback/summary
Analytics	GET /api/clubs/{id}/analytics · GET /api/admin/analytics

11. Working Like a Professional

Git workflow

- Branch per feature: feature/booking-hold, feature/qr-scanner ...
- Commit messages in a standard style: "feat: add seat hold expiry job", "fix: prevent double check-in"
- Merge only when the CI build is green; tag each milestone (v0.1 ... v1.0)

Definition of Done (for every feature)

- It works end to end in the UI, not only in Postman
- Input is validated and errors show friendly messages
- Only the right roles can use it (tested)
- At least one automated test covers the main rule
- Swagger is updated and the README mentions it
- Committed, pushed, CI green

Testing strategy

Level	What	Example
Unit	Service rules with Mockito	Cannot book a CANCELLED event
Integration	API + real MySQL (Testcontainers)	POST booking returns 201 and seats go down
Concurrency	Many threads at once	100 bookings on 10 seats = exactly 10
Frontend	React Testing Library	Countdown shows and disables Pay at 00:00
Manual	Demo script (section 13)	Full journey on a phone

12. Risks & How We Handle Them

Risk	How we handle it
Scope grows and the project never finishes	Strict MUST / SHOULD / COULD list; every milestone is demo-able on its own
Stripe webhooks do not reach localhost	Use the Stripe CLI to forward webhooks to the local backend
Getting stuck on Spring Security	Build it in small steps: in-memory user, then database user, then JWT, then cookie + CSRF
Camera scanner does not work on laptop	Scanner page also has a "type ticket code" box; test on phone over HTTPS after deployment
Losing motivation	Post a short LinkedIn update at every milestone tag

13. The 5-Minute Demo Script (for interviews)

1. Log in as **Organizer (Coding Club)** - create "Hackathon 2026" with 3 seats and submit it.
2. Log in as **Admin** - approve it from the approval queue.
3. As **3 students**, book all seats; the 4th student sees "Sold out - join waitlist (#1)".
4. One student lets the 10-minute hold expire (shortened to 1 minute in demo mode) - the waitlisted student instantly gets an offer email in Mailpit.
5. Pay with the Stripe test card; the QR ticket appears.
6. Open the **Scanner** on a phone - scan: green VALID; scan again: red ALREADY USED.
7. Mark the event completed - download the PDF certificate and open its public verify link.
8. Show the organizer dashboard charts and the green CI pipeline on GitHub.

14. Resume Lines You Will Earn

- Built **EventHub**, a full stack college event platform (React, Spring Boot, MySQL) covering the full event lifecycle from approval to certification.
- Designed a **concurrency-safe booking engine** with timed seat holds and optimistic locking, verified by a 100-thread test with zero overselling.
- Implemented an **automatic waitlist**, **QR-code gate check-in** for volunteers, and **verifiable PDF certificates**.
- Integrated **Stripe Checkout with signed webhooks**, JWT cookie authentication with CSRF protection, Flyway migrations, Testcontainers tests and GitHub Actions CI.

15. Progress Tracker

Milestone	Version	Week	Start date	Finish date	Status
M0 Plain Java warm-up	-	Pre			
M1 Skeleton & pipeline	v0.1	1			
M2 Event catalogue	v0.2	2-3			
M3 Identity, roles, approval	v0.3	4-5			
M4 Booking engine	v0.4	6-7			
M5 Payments	v0.5	8			
M6 Waitlist & notifications	v0.6	9			
M7 QR tickets & check-in	v0.7	10			
M8 Certificates, feedback, analytics	v0.8	11			
M9 Quality & launch	v1.0	12			

Next step: start **M0** - build the plain Java console version of EventHub.